

**Moscow Aviation Institute
(National Research University)**

Faculty of Information Technologies and Applied Mathematics

Department of Computational Mathematics and Programming

Laboratory work No. 4, course “Discrete Analysis”

Student: M. A. Khorokhorin
Teacher: A. A. Kukhtichev
Group: M8O-206BV-24
Date: 11.05.2026
Grade:
Signature:

Moscow, 2026

Laboratory work No. 4

Task: Implement one of the standard pattern matching algorithms for the given alphabet.

Algorithm variant: search for one pattern using the Knuth–Morris–Pratt algorithm.

Alphabet variant: numbers in the range from 0 to $2^{32} - 1$.

Implementing the algorithm on an alphabet of smaller size than specified in the task is not allowed.

Time limit	10 seconds
Memory limit	64 MB
Input	standard input or <code>input.txt</code>
Output	standard output or <code>output.txt</code>

Table 1: Limits of task B. 1-2

Input format. The pattern is given on the first line. Then follows the text consisting of numbers, in which the pattern must be found. There are no limits on line length or on the number of numbers in a line.

Output format. For each occurrence of the pattern, output two numbers separated by a comma: the line number and the word number in the line where the pattern starts. Numbering starts from 1. Line numbers are counted from the start of the text, not including the pattern line.

Example.

Input:

```
11 45 11 45 90
0011 45 011 0045 11 45 90    11
45 11 45 90
```

Output:

```
1, 3
1, 8
```

1 Description

The task is to implement the Knuth–Morris–Pratt algorithm for finding one pattern in a text over the alphabet of numbers from 0 to $2^{32} - 1$.

The KMP algorithm runs in $O(|P| + |T|)$, where $|P|$ is the length of the pattern and $|T|$ is the length of the text. First, the prefix function π is built for the pattern: the value $\pi[i]$ is the length of the longest proper prefix of $P[0..i]$ that is also a suffix of this sequence. Then the text is scanned from left to right. On a mismatch the algorithm never moves back in the text; instead it uses π to choose the next length of the already matched prefix.

Numbers are read into the type `Token`, an alias for `std::uint32_t`, with `std::from_chars`. Numeric parsing handles leading zeros correctly: 0011 and 11 become the same value 11. A token that is not a number or does not fit into $[0, 2^{32} - 1]$ is not truncated silently: the program prints `ERROR: line N: invalid token 'x'` (or `ERROR: pattern: invalid token 'x'` for the pattern line) to the standard error stream and exits with code 1.

2 Source code

1 Prefix function

The function `buildPrefixFunction` builds the array π for the pattern. The variable `length` stores the current length of the matched prefix. While the next symbol does not match, `length` falls back along the already computed values of the prefix function.

2 Matcher

The class `KmpMatcher` holds the pattern, its prefix function and the length of the matched prefix `matched_`. The method `feed(token, position)` processes one token of the text together with its position (a `Position` structure with the fields `line` and `word`) and returns `std::optional<Position>`: the position of the start of an occurrence if the pattern has just been matched, and `std::nullopt` otherwise. To report the start of an occurrence without storing the whole text, the matcher keeps a ring buffer `window_` of the positions of the last $|P|$ tokens. The method `reset()` clears the state before a new text.

Function	Purpose
<code>buildPrefixFunction(pattern)</code>	Builds the prefix function of the pattern in $O(P)$
<code>KmpMatcher(pattern)</code>	Stores the pattern and builds its prefix function
<code>KmpMatcher::feed(token, position)</code>	Processes one token, returns the start of a found occurrence
<code>KmpMatcher::reset()</code>	Resets the matched length and the ring buffer counter
<code>kmpSearch(pattern, text, positions)</code>	Runs <code>KmpMatcher</code> over prepared vectors and returns all occurrences
<code>parseTokens(line)</code>	Splits a line into tokens and checks that each is a valid number
<code>runSearch(input, output)</code>	Streaming search: reads the pattern and the text line by line and prints occurrences

Table 2: Program interface

3 Search

In the main program the text is processed as a stream, line by line and word by word, in `runSearch`. This matters because the statement has no limits on the length of the text, while the memory limit is 64 MB. Each token is passed to `KmpMatcher::feed`, so occurrences that cross line boundaries are found naturally. The function `kmpSearch` is built on the same `KmpMatcher` and is used by the benchmark, so the benchmark measures exactly the code used by the application.

Listing 1: include/kmp.hpp

```
1  #pragma once
2
3  #include <cstddef>
4  #include <cstdint>
5  #include <optional>
6  #include <vector>
7
8  using Token = std::uint32_t;
9
10 struct Position {
11     std::size_t line = 0;
12     std::size_t word = 0;
13
14     bool operator==(const Position& other) const = default;
15 };
16
17 std::vector<std::size_t> buildPrefixFunction(const std::vector<Token>& pattern);
18
19 class KmpMatcher {
20 public:
21     explicit KmpMatcher(std::vector<Token> pattern);
22
23     std::optional<Position> feed(Token token, Position position);
24     void reset();
25
26     [[nodiscard]] const std::vector<Token>& pattern() const { return pattern_; }
27
28 private:
29     std::vector<Token> pattern_;
30     std::vector<std::size_t> prefix_;
31     std::vector<Position> window_;
32     std::size_t fed_ = 0;
33     std::size_t matched_ = 0;
34 };
35
36 std::vector<Position> kmpSearch(const std::vector<Token>& pattern, const std::vector<Token>& text,
37                               const std::vector<Position>& positions);
```

Listing 2: src/kmp.cpp

```
1  #include "kmp.hpp"
2
3  #include <cstddef>
4  #include <optional>
5  #include <utility>
```

```

6  #include <vector>
7
8  std::vector<std::size_t> buildPrefixFunction(const std::vector<Token>& pattern) {
9      std::vector<std::size_t> prefix(pattern.size(), 0);
10     for (std::size_t i = 1; i < pattern.size(); ++i) {
11         std::size_t length = prefix[i - 1];
12         while (length > 0 && pattern[i] != pattern[length]) {
13             length = prefix[length - 1];
14         }
15         if (pattern[i] == pattern[length]) {
16             ++length;
17         }
18         prefix[i] = length;
19     }
20     return prefix;
21 }
22
23 KmpMatcher::KmpMatcher(std::vector<Token> pattern)
24     : pattern_(std::move(pattern)),
25       prefix_(buildPrefixFunction(pattern_)),
26       window_(pattern_.size()) {
27 }
28
29 std::optional<Position> KmpMatcher::feed(Token token, Position position) {
30     if (pattern_.empty()) {
31         return std::nullopt;
32     }
33     window_[fed_ % pattern_.size()] = position;
34     ++fed_;
35
36     while (matched_ > 0 && token != pattern_[matched_]) {
37         matched_ = prefix_[matched_ - 1];
38     }
39     if (token == pattern_[matched_]) {
40         ++matched_;
41     }
42     if (matched_ < pattern_.size()) {
43         return std::nullopt;
44     }
45     matched_ = prefix_[matched_ - 1];
46     return window_[fed_ % pattern_.size()];
47 }
48
49 void KmpMatcher::reset() {
50     fed_ = 0;
51     matched_ = 0;
52 }
53
54 std::vector<Position> kmpSearch(const std::vector<Token>& pattern, const std::vector<Token>& text,
55                               const std::vector<Position>& positions) {
56     std::vector<Position> matches;
57     KmpMatcher matcher(pattern);
58     for (std::size_t i = 0; i < text.size(); ++i) {
59         if (const auto match = matcher.feed(text[i], positions[i])) {
60             matches.push_back(*match);
61         }
62     }
63     return matches;
64 }

```

Listing 3: include/search.hpp

```

1 | #pragma once
2 |
3 | #include <iosfwd>
4 | #include <string_view>
5 | #include <vector>
6 |
7 | #include "kmp.hpp"
8 |
9 | std::vector<Token> parseTokens(std::string_view line);
10 | void runSearch(std::istream& input, std::ostream& output);

```

Listing 4: src/search.cpp

```

1 | #include "search.hpp"
2 |
3 | #include <charconv>
4 | #include <cstdint>
5 | #include <istream>
6 | #include <optional>
7 | #include <ostream>
8 | #include <stdexcept>
9 | #include <string>
10 | #include <string_view>
11 | #include <system_error>
12 | #include <utility>
13 | #include <vector>
14 |
15 | #include "kmp.hpp"
16 |
17 | namespace {
18 |
19 | bool isSpace(char symbol) {
20 |     return symbol == ' ' || symbol == '\t' || symbol == '\r' || symbol == '\n' || symbol == '\v' ||
21 |         symbol == '\f';
22 | }
23 |
24 | Token parseToken(std::string_view text) {
25 |     Token value = 0;
26 |     const auto [end, error] = std::from_chars(text.data(), text.data() + text.size(), value);
27 |     if (error != std::errc{} || end != text.data() + text.size()) {
28 |         throw std::invalid_argument("invalid token '" + std::string(text) + "'");
29 |     }
30 |     return value;
31 | }
32 |
33 | template <typename Callback>
34 | void forEachToken(std::string_view line, const Callback& callback) {
35 |     std::size_t position = 0;
36 |     while (position < line.size()) {
37 |         while (position < line.size() && isSpace(line[position])) {
38 |             ++position;
39 |         }
40 |         const std::size_t start = position;
41 |         while (position < line.size() && !isSpace(line[position])) {
42 |             ++position;
43 |         }
44 |         if (position > start) {
45 |             callback(parseToken(line.substr(start, position - start)));

```

```

46     }
47 }
48 }
49
50 }
51
52 std::vector<Token> parseTokens(std::string_view line) {
53     std::vector<Token> tokens;
54     forEachToken(line, [&tokens](Token token) { tokens.push_back(token); });
55     return tokens;
56 }
57
58 void runSearch(std::istream& input, std::ostream& output) {
59     std::string line;
60     if (!std::getline(input, line)) {
61         return;
62     }
63     std::vector<Token> pattern;
64     try {
65         pattern = parseTokens(line);
66     } catch (const std::invalid_argument& error) {
67         throw std::invalid_argument(std::string("pattern: ") + error.what());
68     }
69     if (pattern.empty()) {
70         return;
71     }
72
73     KmpMatcher matcher(std::move(pattern));
74     std::size_t lineNumber = 0;
75     while (std::getline(input, line)) {
76         ++lineNumber;
77         std::size_t wordNumber = 0;
78         try {
79             forEachToken(line, [&](Token token) {
80                 ++wordNumber;
81                 if (const std::optional<Position> match =
82                     matcher.feed(token, Position{.line = lineNumber, .word = wordNumber})) {
83                     output << match->line << ", " << match->word << '\n';
84                 }
85             });
86         } catch (const std::invalid_argument& error) {
87             throw std::invalid_argument("line " + std::to_string(lineNumber) + ": " + error.what());
88         }
89     }
90 }

```


Listing 5: app/main.cpp

```
1 | #include <exception>
2 | #include <iostream>
3 |
4 | #include "search.hpp"
5 |
6 | int main() {
7 |     std::ios::sync_with_stdio(false);
8 |     std::cin.tie(nullptr);
9 |
10 |     try {
11 |         runSearch(std::cin, std::cout);
12 |     } catch (const std::exception& error) {
13 |         std::cout.flush();
14 |         std::cerr << "ERROR: " << error.what() << '\n';
15 |         return 1;
16 |     }
17 |     return 0;
18 | }
```

3 Console

```
# make build PRESET=release
# build/release/lab4/lab4_main
11 45 11 45 90
0011 45 011 0045 11 45 90    11
45 11 45 90
1, 3
1, 8
# build/release/lab4/lab4_main
1 2
1 2 x 1 2
1, 1
ERROR: line 1: invalid token 'x'
# build/release/lab4/lab4_main
4294967296
ERROR: pattern: invalid token '4294967296'
```

4 Performance test

1 Method

The test compares the implemented Knuth–Morris–Pratt algorithm with naive search. For each position of the text the naive algorithm checks the pattern symbol by symbol, so in the worst case it runs in $O(|P| \cdot |T|)$. KMP builds the prefix function and then scans the text once, which gives $O(|P| + |T|)$. The benchmark measures the same `KmpMatcher` that the application uses (through `kmpSearch`).

Two series of experiments are carried out. In the first series a random text is generated over an alphabet of 10 random numbers, the pattern length is 8. The second series uses the worst case for naive search: the pattern consists of 19 ones followed by a two, and the text consists only of ones.

The search time is measured with `std::chrono::steady_clock`. Data generation and input reading are not included in the measurement. After each run the benchmark compares the full lists of found positions for KMP and the naive algorithm.

2 Results

$ T $	KMP, μs	Naive search, μs
10 000	27	23
100 000	314	293
500 000	951	959
1 000 000	1 813	1 731
5 000 000	10 324	9 712

Table 3: Random text, alphabet of 10 numbers, pattern length 8

$ T $	KMP, μs	Naive search, μs
100 000	1 610	2 310
500 000	1 186	5 644
1 000 000	2 708	10 895
5 000 000	11 924	55 733

Table 4: Worst case for naive search

```
# make build PRESET=release
# for n in 10000 100000 500000 1000000 5000000; do
>   echo "n=$n:"
```

```

> python3 lab4/benchmark/generator.py $n 8 42 10
> build/release/lab4/lab4_benchmark < lab4/benchmark/tests.txt
> done
n=10000:
KMP time: 27us
Naive search time: 23us
KMP matches: 0
Naive matches: 0
Results equal: yes
n=100000:
KMP time: 314us
Naive search time: 293us
KMP matches: 0
Naive matches: 0
Results equal: yes
n=500000:
KMP time: 951us
Naive search time: 959us
KMP matches: 0
Naive matches: 0
Results equal: yes
n=1000000:
KMP time: 1813us
Naive search time: 1731us
KMP matches: 0
Naive matches: 0
Results equal: yes
n=5000000:
KMP time: 10324us
Naive search time: 9712us
KMP matches: 0
Naive matches: 0
Results equal: yes

# for n in 100000 500000 1000000 5000000; do
> echo "n=$n:"
> python3 lab4/benchmark/generator_worst.py $n
> build/release/lab4/lab4_benchmark < lab4/benchmark/tests.txt
> done
n=100000:
KMP time: 1610us

```

```

Naive search time: 2310us
KMP matches: 0
Naive matches: 0
Results equal: yes
n=500000:
KMP time: 1186us
Naive search time: 5644us
KMP matches: 0
Naive matches: 0
Results equal: yes
n=1000000:
KMP time: 2708us
Naive search time: 10895us
KMP matches: 0
Naive matches: 0
Results equal: yes
n=5000000:
KMP time: 11924us
Naive search time: 55733us
KMP matches: 0
Naive matches: 0
Results equal: yes

```

3 Analysis of results

On random data naive search usually stops checking after the first or second number of the pattern, so its practical running time is close to linear. Because of this KMP gets no noticeable advantage: in some runs it is slightly faster, in others slightly slower due to the additional accesses to the prefix function.

In the worst case the difference becomes significant. For $|T| = 5\,000\,000$ the naive algorithm takes **55 733 μ s**, while KMP takes **11 924 μ s**, that is, KMP is about **4.7 times** faster. This agrees with the theoretical difference between $O(|P| \cdot |T|)$ and $O(|P| + |T|)$. The KMP time for $|T| = 100\,000$ is higher than for 500 000: a run of about a millisecond is dominated by measurement noise (cold caches and the first access to freshly allocated memory), so the comparison relies on the larger sizes.

5 Work diary

- 11.05.2026: studied the task of searching for one pattern in a text over the alphabet of numbers from 0 to $2^{32} - 1$.
- 11.05.2026: implemented the construction of the prefix function and the streaming Knuth–Morris–Pratt algorithm that does not store the whole text in memory.
- 11.05.2026: prepared the main generator, the worst-case generator and the benchmark for comparison with naive search.
- 11.05.2026: checked overlapping occurrences, occurrences crossing line boundaries, leading zeros in numbers and conformance of the output to the contest format.

6 Conclusions

In this laboratory work for the course “Discrete Analysis” I implemented the Knuth–Morris–Pratt algorithm for finding a pattern in a text over the alphabet of integers. I studied the construction of the prefix function and the fallback mechanism that avoids repeated comparisons. I implemented correct handling of leading zeros by parsing numbers into `uint32_t`, reporting of invalid tokens instead of silent truncation, and streaming search for occurrences that cross line boundaries without storing the whole text in memory, using `KmpMatcher` with a ring buffer of the last $|P|$ positions.

Testing confirmed the theoretical estimates: on random data the advantage of KMP is not always noticeable because partial matches are short, but on pathological inputs (a pattern of 19 ones followed by a two in a text of ones) the algorithm is about $4.7\times$ faster than naive search for $|T| = 5\,000\,000$, which illustrates the difference between $O(|P| + |T|)$ and $O(|P| \cdot |T|)$.

References

- [1] Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. *Introduction to Algorithms, 2nd edition*. — MIT Press, 2001.
- [2] Knuth D. E., Morris J. H., Pratt V. R. *Fast Pattern Matching in Strings*. SIAM Journal on Computing, 6(2):323–350, 1977.
- [3] Sedgewick R. *Algorithms in C++, Parts 1–4: Fundamentals, Data Structures, Sorting, Searching, 3rd edition*. — Addison-Wesley, 1998.